

Formation Engine Methodology

Section 7 — Statistical Interpretation Layer

Formation Engine Development Team

Version 3.0.0 — As Implemented in Codebase

1 Introduction: From Trajectories to Conclusions

The formation engine produces numbers: energies, forces, positions, velocities. The statistical interpretation layer converts these numbers into **scientific conclusions**: "Has the system converged?" "Are these two structures the same?" "Which formation is more stable?"

This section documents the **actual implementations** of:

1. Welford's algorithm for numerically stable variance computation
2. Convergence detection via stationarity gates
3. Kabsch algorithm for structural alignment and RMSD
4. Observable tracking across trajectories

Every algorithm is documented with file locations, line numbers, and validation criteria.

2 Welford's Algorithm for Online Statistics

Status: Referenced in Section 4.6 and Section 6.6, but **not explicitly implemented** in the atomistic integrators. The concept is documented here for completeness and future implementation.

2.1 The Numerical Stability Problem

The naive formula for variance:

$$\text{Var}(x) = \overline{x^2} - \bar{x}^2 = \frac{1}{n} \sum_{i=1}^n x_i^2 - \left(\frac{1}{n} \sum_{i=1}^n x_i \right)^2 \quad (1)$$

is catastrophically unstable when $\overline{x^2}$ and $\bar{x}^2$ are both large and nearly equal.

Example: For energy trajectories with values $E = -500.001, -500.002, -499.998, \dots$ kcal/mol:

$$\overline{E^2} \approx 250,000 \quad (2)$$

$$\bar{E}^2 \approx 250,000 \quad (3)$$

$$\text{Var}(E) = 250,000 - 250,000 + \varepsilon \quad (\text{catastrophic cancellation}) \quad (4)$$

The subtraction loses precision, and small numerical errors dominate the result.

2.2 Welford's One-Pass Algorithm

Welford (1962) derived a recurrence relation that avoids the catastrophic subtraction:

$$\bar{x}_n = \bar{x}_{n-1} + \frac{x_n - \bar{x}_{n-1}}{n} \quad (5)$$

$$M_{2,n} = M_{2,n-1} + (x_n - \bar{x}_{n-1})(x_n - \bar{x}_n) \quad (6)$$

$$\text{Var}(x) = \frac{M_2}{n-1} \quad (7)$$

The key property: variance is accumulated through products of *deviations from the running mean*, which are small even when the values themselves are large.

2.3 Reference Implementation (Python)

From tools/scoring.py (conceptual, not used in C++ integrators):

```
class OnlineStats:
    """Welford's online algorithm for mean and variance"""
    def __init__(self):
        self.n = 0
        self.mean = 0.0
        self.M2 = 0.0

    def update(self, x):
        self.n += 1
        delta = x - self.mean
        self.mean += delta / self.n
        delta2 = x - self.mean
        self.M2 += delta * delta2

    @property
    def variance(self):
        return self.M2 / (self.n - 1) if self.n > 1 else 0.0

    @property
    def stddev(self):
        return self.variance ** 0.5
```

2.4 Extension to Vectors

For 3D positions and velocities, the algorithm extends component-wise:

```
class OnlineVec3Stats:
    """Per-component statistics for 3D vectors"""
    def __init__(self):
        self.stats_x = OnlineStats()
        self.stats_y = OnlineStats()
        self.stats_z = OnlineStats()

    def update(self, v: Vec3):
        self.stats_x.update(v.x)
        self.stats_y.update(v.y)
```

```

        self.stats_z.update(v.z)

@property
def mean(self) -> Vec3:
    return Vec3(
        self.stats_x.mean,
        self.stats_y.mean,
        self.stats_z.mean
    )

@property
def variance(self) -> Vec3:
    return Vec3(
        self.stats_x.variance,
        self.stats_y.variance,
        self.stats_z.variance
    )

```

2.5 Current Implementation Status

Not implemented in C++: The velocity Verlet and FIRE integrators do not currently track running statistics. Energy and temperature are computed at each step but not accumulated via Welford’s algorithm.

Future work: Integrate `OnlineStats` into the integrators to monitor:

- Energy variance (for heat capacity estimation, Section 4.6)
- Temperature fluctuations (for equilibration detection)
- Force magnitude variance (for convergence diagnostics)

3 Convergence Detection

Status: Conceptual framework defined in Section 6.6, but **not automated** in the integrators.

3.1 The Stationarity Gate

A stationarity gate determines when an observable has reached statistical equilibrium. The test is:

$$|x_{\text{new}} - \bar{x}| < 3\sigma + \varepsilon_{\text{mean}} \quad (8)$$

where:

- $\bar{x}$ is the running mean from Welford’s algorithm
- σ is the running standard deviation
- $\varepsilon_{\text{mean}}$ is a small tolerance for near-zero variance (default: 10^{-6})

Interpretation: A new sample is ”consistent” if it falls within three standard deviations of the running mean (99.7% of samples in a Gaussian distribution).

3.2 Consecutive-Pass Requirement

Convergence requires k **consecutive** passes (default $k = 10$). A single outlier resets the counter.

Rationale: This is conservative by design. It prevents premature convergence declaration from a lucky sequence of low-variance samples. It is better to run 100 extra steps than to report an unstable formation as converged.

3.3 Multi-Observable Tracking

The `ObservableTracker` (not yet implemented) would monitor multiple observables simultaneously:

```
struct ObservableTracker {
    OnlineStats energy_total;
    OnlineStats energy_bond;
    OnlineStats energy_vdW;
    OnlineStats energy_Coulomb;
    OnlineStats temperature;

    int consecutive_passes = 0;
    int required_passes = 10;

    bool update(const State& s, const EnergyTerms& E, double T) {
        energy_total.update(E.total());
        energy_bond.update(E.Ubond);
        energy_vdW.update(E.UvdW);
        energy_Coulomb.update(E.UCoul);
        temperature.update(T);

        // Check all observables
        bool all_pass = true;
        all_pass &= is_stationary(E.total(), energy_total);
        all_pass &= is_stationary(T, temperature);

        if (all_pass) {
            consecutive_passes++;
        } else {
            consecutive_passes = 0;
        }

        return consecutive_passes >= required_passes;
    }

private:
    bool is_stationary(double x_new, const OnlineStats& stats) {
        if (stats.n < 10) return false; // Need minimum samples
        double dev = std::abs(x_new - stats.mean);
        return dev < 3.0 * stats.stddev + 1e-6;
    }
};
```

3.4 Practical Application

MD equilibration: Run Langevin dynamics until energy and temperature converge (typically 1000–5000 steps).

Basin probing: Perturb a structure, minimize with FIRE, record energy. Repeat until the energy distribution converges (typically 50–200 samples).

Conformer sampling: Generate random initial configurations, minimize each, cluster by RMSD. Continue until no new conformers are found for k consecutive samples.

4 Kabsch Algorithm for Structural Alignment

Implementation: `atomistic/core/alignment.cpp`, function `kabsch_align`

The Kabsch algorithm finds the optimal rigid-body rotation to minimize RMSD between two molecular structures.

4.1 The RMSD Problem

Given two structures with N atoms each:

- Reference: $\{\mathbf{r}_i\}_{i=1}^N$
- Target: $\{\mathbf{x}_i\}_{i=1}^N$

Find the rotation matrix $\mathbf{R}$ that minimizes:

$$\text{RMSD}(\mathbf{R}) = \sqrt{\frac{1}{N} \sum_{i=1}^N |\mathbf{R}\mathbf{x}_i - \mathbf{r}_i|^2} \quad (9)$$

4.2 The Kabsch Solution

Kabsch (1976, 1978) proved that the optimal rotation is:

$$\mathbf{R}^* = \mathbf{V}\mathbf{U}^T \quad (10)$$

where $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ is the singular value decomposition (SVD) of the cross-covariance matrix:

$$\mathbf{H} = \sum_{i=1}^N \mathbf{x}_i \otimes \mathbf{r}_i = \begin{bmatrix} \sum x_i r_i^x & \sum x_i r_i^y & \sum x_i r_i^z \\ \sum y_i r_i^x & \sum y_i r_i^y & \sum y_i r_i^z \\ \sum z_i r_i^x & \sum z_i r_i^y & \sum z_i r_i^z \end{bmatrix} \quad (11)$$

4.3 Implementation

From `alignment.cpp`, lines 79–153:

```
AlignmentResult kabsch_align(State& target, const State& reference) {
    AlignmentResult result;

    // Store initial COM positions
    result.target_com_before = compute_com(target);
```

```

result.reference_com = compute_com(reference);

result.rmsd_before = compute_rmsd(target, reference);

if (target.N != reference.N || target.N < 2) {
    // Identity rotation for invalid input
    result.R = linalg::Mat3::identity();
    result.translation = {0, 0, 0};
    result.rmsd_after = result.rmsd_before;
    result.target_com_after = result.target_com_before;
    result.max_deviation = 0.0;
    return result;
}

// Center both states at COM
State ref_copy = reference;
center_at_origin(target);
center_at_origin(ref_copy);

// Build covariance matrix  $H = (\text{target}_i \text{ reference}_i)$ 
linalg::Mat3 H = linalg::Mat3::zero();
for (uint32_t k = 0; k < target.N; ++k) {
    H(0,0) += target.X[k].x * ref_copy.X[k].x;
    H(0,1) += target.X[k].x * ref_copy.X[k].y;
    H(0,2) += target.X[k].x * ref_copy.X[k].z;
    H(1,0) += target.X[k].y * ref_copy.X[k].x;
    H(1,1) += target.X[k].y * ref_copy.X[k].y;
    H(1,2) += target.X[k].y * ref_copy.X[k].z;
    H(2,0) += target.X[k].z * ref_copy.X[k].x;
    H(2,1) += target.X[k].z * ref_copy.X[k].y;
    H(2,2) += target.X[k].z * ref_copy.X[k].z;
}

// SVD:  $H = U V^T$ 
linalg::SVD3 svd(H);

// Optimal rotation:  $R = VU^T$ 
linalg::Mat3 R_opt = svd.V * svd.U.transpose();

// Chirality check: if  $\det(R) < 0$ , reflection occurred
if (R_opt.det() < 0) {
    // Flip last column of V (smallest singular value)
    svd.V(0,2) = -svd.V(0,2);
    svd.V(1,2) = -svd.V(1,2);
    svd.V(2,2) = -svd.V(2,2);
    R_opt = svd.V * svd.U.transpose();
}

// Track maximum deviation
result.max_deviation = 0.0;
for (uint32_t i = 0; i < target.N; ++i) {
    Vec3 before = target.X[i];
    Vec3 after = R_opt * target.X[i];
    Vec3 displacement = after - before;

```

```

        double dev = std::sqrt(dot(displacement, displacement));
        if (dev > result.max_deviation) {
            result.max_deviation = dev;
        }
    }

    // Apply rotation to target
    for (uint32_t i = 0; i < target.N; ++i) {
        target.X[i] = R_opt * target.X[i];
    }

    // Also rotate velocities if present
    if (target.V.size() == target.N) {
        for (uint32_t i = 0; i < target.N; ++i) {
            target.V[i] = R_opt * target.V[i];
        }
    }

    // Store result
    result.R = R_opt;
    result.translation = {0, 0, 0};
    result.target_com_after = compute_com(target);
    result.rmsd_after = compute_rmsd(target, ref_copy);

    return result;
}

```

4.4 Chirality Correction

The line `if (R_opt.det() < 0)` implements the chirality check (lines 117–122).

Physical meaning: If $\det(\mathbf{R}) < 0$, the matrix represents a reflection (improper rotation), not a pure rotation. This occurs when the structures have opposite chirality (e.g., L vs. D amino acids).

Correction: Flip the sign of the column of $\mathbf{V}$ corresponding to the smallest singular value. This changes the reflection into a proper rotation while minimizing the change to the RMSD.

4.5 SVD Implementation

From `atomistic/core/linalg.hpp`, struct `SVD3` (lines 98–113):

```

/**
 * SVD decomposition for 3x3 matrix:  $A = U V^T$ 
 * Uses Jacobi iteration for symmetric eigenvalue problem
 *
 * Algorithm:
 * 1. Form  $A^T A$  (symmetric 3x3)
 * 2. Diagonalize via Jacobi rotations  $V$  and
 * 3. Compute  $U = A V^{-1}$ 
 *
 * Accuracy:  $\sim 1e-12$  for well-conditioned matrices
 */
struct SVD3 {

```

```

Mat3 U;          // Left singular vectors (3x3 orthogonal)
Vec3 sigma;      // Singular values (      0)
Mat3 V;          // Right singular vectors (3x3 orthogonal)

explicit SVD3(const Mat3& A);
};

```

The SVD is computed via Jacobi iteration, a classical method for symmetric eigenvalue problems. The algorithm:

1. Form $\mathbf{A}^T \mathbf{A}$ (symmetric positive semi-definite)
2. Apply Jacobi rotations to diagonalize $\mathbf{A}^T \mathbf{A}$, yielding $\mathbf{V}$ and $\mathbf{\Sigma}^2$
3. Compute $\mathbf{U} = \mathbf{A} \mathbf{V} \mathbf{\Sigma}^{-1}$

Accuracy: Jacobi iteration converges to machine precision ($\sim 10^{-12}$) for well-conditioned matrices. For ill-conditioned matrices (e.g., colinear atoms), the smallest singular value approaches zero, but the algorithm remains stable.

4.6 RMSD Computation

From `alignment.cpp`, lines 53–62:

```

double compute_rmsd(const State& a, const State& b) {
    if (a.N != b.N || a.N == 0) return 0.0;

    double sum = 0.0;
    for (uint32_t i = 0; i < a.N; ++i) {
        Vec3 d = a.X[i] - b.X[i];
        sum += dot(d, d);
    }

    return std::sqrt(sum / a.N);
}

```

This is the textbook definition:

$$\text{RMSD} = \sqrt{\frac{1}{N} \sum_{i=1}^N |\mathbf{x}_i - \mathbf{r}_i|^2} \quad (12)$$

4.7 Usage in Conformer Clustering

Protocol:

1. Generate M conformers via perturbation + FIRE minimization
2. For each pair (i, j) :
 - (a) Align structure i onto structure j via Kabsch
 - (b) Compute RMSD_{ij} after alignment
3. Cluster by RMSD: conformers with $\text{RMSD} < 0.5 \text{ \AA}$ are the same

RMSD threshold: The default of 0.5 Å is a heuristic. For small molecules, 0.3 Å may be more appropriate. For proteins, 1.0–2.0 Å is common.

5 Animated Alignment

Implementation: `atomistic/core/alignment.cpp`, function `animated_align`

For visualization purposes, the alignment can be animated smoothly over N steps (default: 60).

5.1 The Animation Loop

From `alignment.cpp`, lines 179–210 (partial):

```
AlignmentResult animated_align(
    State& target,
    const State& reference,
    int n_steps,
    std::optional<std::function<void(double, double, const State&)>> callback
) {
    // Store initial state
    State initial_target = target;

    // Compute final alignment
    State temp_target = target;
    AlignmentResult final_result = kabsch_align(temp_target, reference);

    // Restore target to initial state
    target = initial_target;
    center_at_origin(target);

    // Interpolate rotation from identity to final R
    Mat3 R_identity = Mat3::identity();
    Mat3 R_final = final_result.R;

    for (int step = 0; step <= n_steps; ++step) {
        double t = (double)step / (double)n_steps;

        // Linear interpolation (not ideal, but simple)
        Mat3 R_t = interpolate_rotation(R_identity, R_final, t);

        // Apply partial rotation
        State rotated_target = initial_target;
        for (uint32_t i = 0; i < rotated_target.N; ++i) {
            rotated_target.X[i] = R_t * rotated_target.X[i];
        }

        // Compute current RMSD
        double rmsd_current = compute_rmsd(rotated_target, reference);

        // Call visualization callback
        if (callback.has_value()) {
            (*callback)(t, rmsd_current, rotated_target);
        }
    }
}
```

```

    return final_result;
}

```

5.2 Rotation Interpolation

The current implementation uses **linear interpolation** of matrix elements:

$$\mathbf{R}(t) = (1 - t)\mathbf{I} + t\mathbf{R}_{\text{final}} \quad (13)$$

Limitation: This is *not* a proper rotation for $t \in (0, 1)$. The interpolated matrix may not be orthogonal.

Better approach: Use quaternion representation and SLERP (spherical linear interpolation):

1. Convert $\mathbf{R}_{\text{final}}$ to quaternion $\mathbf{q}_{\text{final}}$
2. Interpolate: $\mathbf{q}(t) = \text{slerp}(\mathbf{q}_{\text{identity}}, \mathbf{q}_{\text{final}}, t)$
3. Convert $\mathbf{q}(t)$ back to rotation matrix

This ensures smooth, physically meaningful rotations at all intermediate steps.

5.3 Visualization Callback

The callback function signature:

```
std::function<void(double progress, double rmsd, const State& current)>
```

Usage: Update camera position, render frame, display RMSD value on screen. This enables real-time visualization of the alignment process.

6 Observable Tracking Across Trajectories

Status: Conceptual framework only. Not implemented in integrators.

6.1 Observable Types

Observable	Units	Typical Fluctuation
Total energy E	kcal/mol	± 0.1 kcal/mol per atom
Temperature T	K	$\pm \sqrt{2/(3N)}$ relative
Bond length	Å	± 0.01 – 0.05 Å
Angle	degrees	± 1 – 5°
Dihedral	degrees	± 5 – 20° (flexible)
RMSD from initial	Å	System-dependent
Radius of gyration R_g	Å	± 0.1 – 1.0 Å

6.2 Time-Series Analysis

Autocorrelation function: Measures correlation between a variable at time t and time $t + \tau$:

$$C(\tau) = \frac{\langle (x(t) - \bar{x})(x(t + \tau) - \bar{x}) \rangle}{\langle (x(t) - \bar{x})^2 \rangle} \quad (14)$$

Correlation time: The time τ_c at which $C(\tau_c) = 1/e \approx 0.368$. Samples separated by more than τ_c are approximately independent.

Effective sample size: For N total samples with correlation time τ_c (measured in steps):

$$N_{\text{eff}} = \frac{N}{\tau_c} \quad (15)$$

This is the number of independent samples, used to correct variance estimates.

6.3 Block Averaging

An alternative to autocorrelation analysis: divide the trajectory into M blocks of length L , compute the mean of each block, then estimate variance from block means:

$$\bar{x}_m = \frac{1}{L} \sum_{i=1}^L x_{(m-1)L+i} \quad \text{for } m = 1, \dots, M \quad (16)$$

$$\text{Var}_{\text{block}} = \frac{1}{M-1} \sum_{m=1}^M (\bar{x}_m - \bar{x})^2 \quad (17)$$

If $L \gg \tau_c$, the block means are approximately independent, and $\text{Var}_{\text{block}}$ correctly estimates the variance of $\bar{x}$.

7 Scoring and Classification Integration

Implementation: `tools/scoring.py` and `tools/classification.py` (documented in Section 6)

The statistical interpretation layer feeds into the scoring system:

1. **Convergence status** determines w_S (stability gate):
 - Converged (stationarity gate passed): $w_S = 1.0$
 - Bounded but not converged: $w_S = 0.3$
 - Exploded (NaN, divergence): $w_S = 0.05$
2. **Energy variance** estimates formation stability:
 - Low variance ($\sigma_E < 0.1$ kcal/mol): Single deep basin
 - High variance ($\sigma_E > 1.0$ kcal/mol): Multiple competing basins
3. **RMSD clustering** groups conformers:
 - All structures with $\text{RMSD} < 0.5 \text{ \AA}$ belong to the same conformer
 - Different conformers are ranked by energy

8 Validation and Sanity Checks

8.1 Kabsch Validation

Test 1: Rotation invariance

Generate a random rotation $\mathbf{R}_{\text{test}}$, apply it to a structure:

$$\mathbf{x}'_i = \mathbf{R}_{\text{test}} \mathbf{x}_i \quad (18)$$

Align the rotated structure back onto the original:

$$\mathbf{R}_{\text{Kabsch}} = \text{kabsch}(\mathbf{x}', \mathbf{x}) \quad (19)$$

Expected: $\mathbf{R}_{\text{Kabsch}} = \mathbf{R}_{\text{test}}^{-1}$ to machine precision.

Test 2: RMSD minimization

For a pair of structures, compute RMSD before and after alignment:

$$\text{RMSD}_{\text{before}} = \text{compute_rmsd}(\text{target}, \text{reference}) \quad (20)$$

$$\text{RMSD}_{\text{after}} = \text{compute_rmsd}(\text{aligned_target}, \text{reference}) \quad (21)$$

Expected: $\text{RMSD}_{\text{after}} \leq \text{RMSD}_{\text{before}}$ (alignment can only improve or maintain RMSD, never worsen it).

Test 3: Chirality preservation

For a chiral molecule (e.g., L-alanine), align two copies:

$$\det(\mathbf{R}_{\text{Kabsch}}) > 0 \quad (22)$$

For mirror images (L-alanine vs. D-alanine):

$$\det(\mathbf{R}_{\text{Kabsch}}) < 0 \quad (\text{reflection required}) \quad (23)$$

After chirality correction (line 120):

$$\det(\mathbf{R}_{\text{corrected}}) > 0 \quad (24)$$

8.2 Convergence Detection Validation

Test: Generate synthetic trajectory:

$$E(t) = E_0 + \sigma \cdot \mathcal{N}(0, 1) + A \sin(\omega t) \quad (25)$$

with equilibrium energy $E_0 = -100$ kcal/mol, noise $\sigma = 0.1$ kcal/mol, oscillation amplitude $A = 0.5$ kcal/mol.

Expected behavior:

- Stationarity gate should pass after ~ 50 samples (3 periods of oscillation)
- Running mean should converge to $E_0 \pm 0.01$ kcal/mol
- Running variance should converge to $\sigma^2 + A^2/2 = 0.135$ (kcal/mol)²

9 Known Limitations and Future Work

9.1 Welford’s Algorithm: Not Integrated

- Velocity Verlet and FIRE do not use Welford’s algorithm
- Energy and temperature are computed at each step but not accumulated
- Variance estimates are not available during simulation

Future work: Add `OnlineStats` to integrator state, track energy/temperature variance in real time.

9.2 Convergence Detection: Manual

- No automatic convergence detection in integrators
- User must manually inspect energy/temperature trajectories
- Stationarity gate is a conceptual tool, not implemented

Future work: Integrate `ObservableTracker` into Langevin dynamics, auto-stop when converged.

9.3 Kabsch Alignment: Single Reference

- Current implementation aligns target onto a single reference
- No support for multiple-reference alignment (e.g., align to ensemble average)
- No support for partial alignment (e.g., align backbone only, leave sidechains free)

Future work: Weighted Kabsch (align subsets), iterative refinement for ensemble averaging.

9.4 Animation: Linear Interpolation

- Rotation interpolation is not geodesic (shortest path on $SO(3)$)
- Intermediate matrices are not proper rotations
- Visual artifacts may occur for large rotations

Future work: Implement SLERP (spherical linear interpolation) via quaternions.

10 Conclusion: Statistical Rigor as Foundation

This section has documented the statistical interpretation layer **as it exists in the codebase**:

- **Welford’s algorithm:** Conceptual framework defined, not implemented in integrators
- **Convergence detection:** Stationarity gate documented, not automated
- **Kabsch alignment:** Fully implemented in `atomistic/core/alignment.cpp` with SVD via Jacobi iteration

- **RMSD computation:** Textbook definition, validated for conformer clustering
- **Animated alignment:** Functional with linear interpolation (SLERP upgrade pending)

The Kabsch implementation is production-ready. The convergence detection and variance tracking are conceptual frameworks awaiting integration.

The formation engine’s commitment to **reproducibility and auditability** (Section 11) requires that every statistical conclusion be traceable to raw data. Welford’s algorithm and convergence detection are the mathematical tools that convert trajectories into defensible conclusions.

The principle: If you cannot compute the variance of an observable, you cannot claim it has converged. If you cannot align two structures optimally, you cannot claim they are different. Statistical rigor is not optional—it is the foundation of scientific validity.

Transition to §8–9: With the statistical tools established (variance estimation, convergence detection, structural alignment), Sections 8 and 9 tackle the next frontier: predicting *electronic properties* (charges, reactivity indices) and *chemical reactions* (site matching, barrier estimation) from the classical framework.